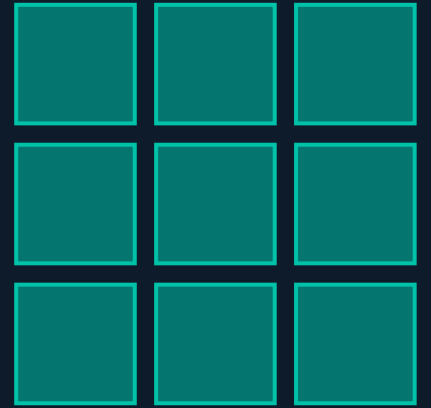


SUDOKU IN C++

A step-by-step build using Object-Oriented Programming, Data Structures & Algorithms

Walkthrough • 8 Classes • 8 Test Cases



Presented by:

- **Zeinab Ayman**
- **Wafaa El-Manzalawy**

The Build Order

Each class builds on the one before it — implement (and test!) in this order.

1

Cell

The atomic unit: one value, one fixed-flag

2

Board

Owns the 9x9 grid; enforces Sudoku rules

3

SudokuSolver

Backtracking algorithm that solves any board

4

SudokuGenerator

Builds a puzzle with a unique solution

5

MoveHistory

Two stacks power undo / redo

6

GameController

Coordinates every class above

7

FileManager

Serializes game state to disk

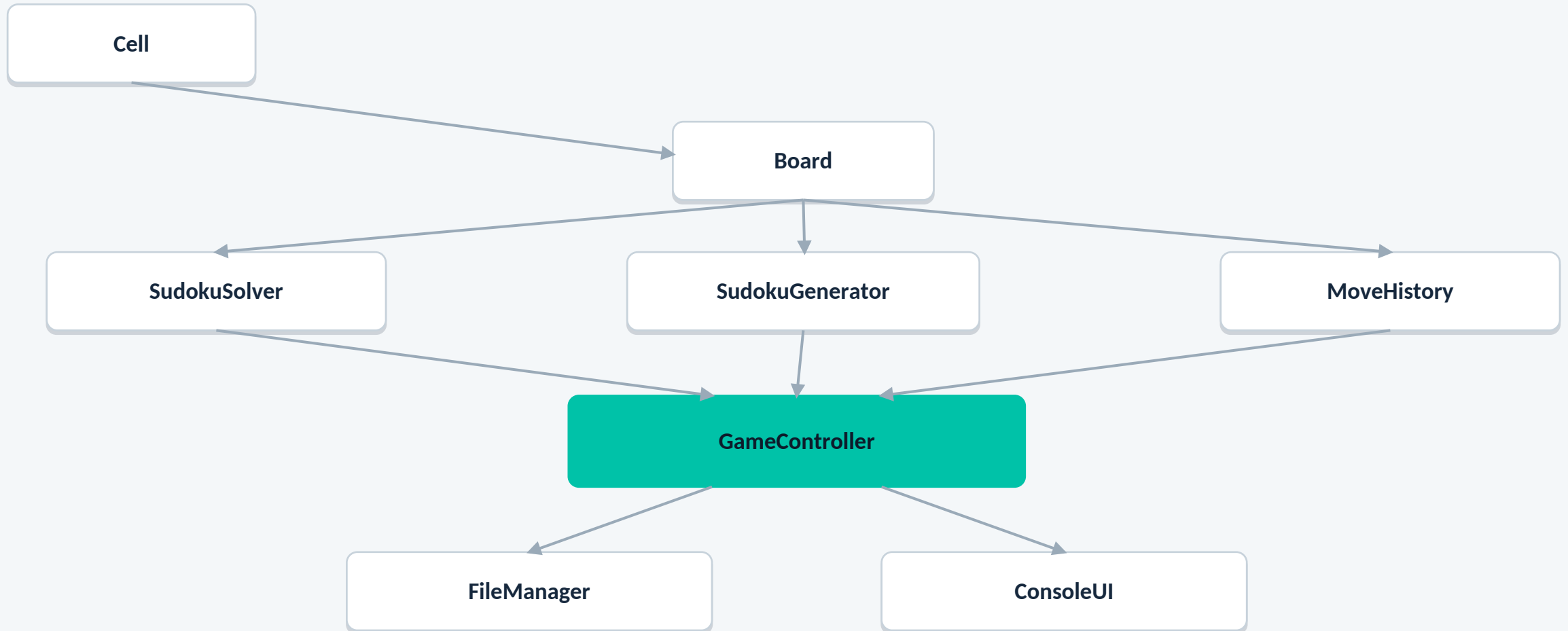
8

ConsoleUI

Menus, board rendering, the game loop

Architecture at a Glance

Data flows up from the smallest unit; control flows down from the GameController.



1

Cell

The smallest unit on the board — one value, one flag.

STEP 1 / 8

WHAT IT OWNS

- An int value: 0 means empty, 1-9 is a filled digit
- A bool isFixed flag: true for original puzzle clues
- A set of candidate values (1-9), used later for hints
- No knowledge of rows, columns, or boxes — Board owns that

```
class Cell {  
public:  
    int getValue() const;  
    void setValue(int v);  
    bool isEmpty() const;  
    bool isFixed() const;  
    void setFixed(bool f);  
private:  
    int value_ = 0;  
    bool isFixed_ = false;  
};
```

TEST CASE

INPUT

```
Cell c;  
c.setValue(5);  
c.setFixed(true);  
cout << c.getValue();  
cout << c.isFixed();
```

EXPECTED OUTPUT

```
5  
1
```



Board

Owns the 9x9 grid and enforces Sudoku's rules.

STEP 2 / 8

WHAT IT OWNS

- A 9x9 array of Cell objects
- isValid / isColValid / isBoxValid: rule checks
- isPlacementValid: combines all three checks
- isComplete: true once every cell is filled correctly

```
bool Board::isValid(
    int row, int value) const {
    for (int c = 0; c < SIZE; c++) {
        if (grid_[row][c].getValue()
            == value) {
            return false;
        }
    }
    return true;
}
```

TEST CASE

INPUT

```
Board b;
b.at(0,0).setValue(5);
```

```
b.isValid(0, 5);
b.isValid(0, 7);
```

EXPECTED OUTPUT

```
false    (5 is already in row 0)
true     (7 is still free)
```

3

SudokuSolver

Backtracking: try, recurse, undo, retry.

STEP 3 / 8

THE ALGORITHM

- Find the next empty cell — none left means solved
- Try values 1-9 in that cell
- If a value is valid, place it and recurse
- If the recursive call fails, undo (backtrack) and try the next value
- countSolutions() reuses this to prove a puzzle is unique

```
for (int v = 1; v <= 9; v++) {  
    if (board.isPlacementValid(  
        row, col, v)) {  
        board.at(row,col).setValue(v);  
        if (solveRecursive(board))  
            return true;  
        board.at(row,col).clearValue();  
    }  
}  
return false; // backtrack
```

TEST CASE

INPUT

```
Board b;  
SudokuSolver solver;  
  
bool ok = solver.solve(b);
```

EXPECTED OUTPUT

```
ok == true, and every row / column /  
3x3 box contains 1-9 exactly once
```

4

SudokuGenerator

A solved board, minus just enough cells.

STEP 4 / 8

THE STEPS

- Fill the 3 diagonal boxes with shuffled 1-9 (they never collide)
- Fill the rest with the same backtracking, but randomized order
- Remove cells one at a time, difficulty sets how many
- After each removal, re-check `countSolutions() == 1`
- If not unique, put the value back and try a different cell

```
int before = board.at(r,c).getValue();  
board.at(r,c).clearValue();
```

```
Board copy = board;  
if (solver_.countSolutions(copy, 2)  
    != 1) {  
    board.at(r,c).setValue(before);  
    // undo: keep the puzzle unique  
}
```

TEST CASE

INPUT

```
SudokuGenerator gen;  
Board puzzle =  
    gen.generate(Difficulty::EASY);  
  
solver.countSolutions(puzzle, 2);
```

EXPECTED OUTPUT

1 (exactly one solution)

HOW IT WORKS

- Every move is pushed onto the undo stack
- undo(): pop from undo, read it, push onto redo
- redo(): pop from redo, read it, push back onto undo
- A brand-new move clears the redo stack entirely

```
bool MoveHistory::undo(Move& out) {  
    if (undoStack_.empty())  
        return false;  
    out = undoStack_.top();  
    undoStack_.pop();  
    redoStack_.push(out);  
    return true;  
}
```

TEST CASE

INPUT

```
MoveHistory h;  
h.recordMove({0,0,5,7});  
Move m;  
h.undo(m);  
h.canRedo();
```

EXPECTED OUTPUT

```
m.previousValue == 5  
true
```

6

GameController

The orchestrator — the UI only talks to this.

STEP 6 / 8

WHAT IT COORDINATES

- `startNewGame()`: generate a puzzle + compute the full solution
- `makeMove()`: reject fixed cells, else apply + record the move
- `undo()` / `redo()`: apply the value MoveHistory hands back
- `giveHint()` / `solvePuzzle()`: read straight from the solution

```
bool GameController::makeMove(
    int row, int col, int value) {
    if (board_.at(row,col).isFixed())
        return false;
    int old =
        board_.at(row,col).getValue();
    board_.at(row,col).setValue(value);
    history_.recordMove(
        {row, col, old, value});
    return true;
}
```

TEST CASE

INPUT

```
GameController gc;
gc.startNewGame(Difficulty::EASY);
gc.makeMove(0, 0, 9);
gc.makeMove(fixedRow, fixedCol, 3);
```

EXPECTED OUTPUT

true or false, depending on the rules
always false (clue cells are locked)

7

FileManager

Save and load — same order in, same order out.

STEP 7 / 8

THE FORMAT

- 9 lines of cell values (0 = empty)
- 9 lines of fixed-cell flags (0 / 1)
- 1 line of stats: time, moves, mistakes, hints
- loadFromFile() reads back in the exact same order

```
for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
        out << board.at(r,c).getValue()
            << " ";
    }
    out << "\n";
}
// repeat for fixed flags, then
// write the stats line
```

TEST CASE

INPUT

```
Board b; b.at(0,0).setValue(7);
FileManager::saveToFile(
    "test.txt", b, 10, 2, 0, 1);
Board b2; FileManager::loadFromFile(
    "test.txt", b2, t, m, mi, h);
```

EXPECTED OUTPUT

```
b2.at(0,0).getValue() == 7
```

8

ConsoleUI

Menus, the board on screen, and the game loop.

STEP 8 / 8

WHAT IT DOES

- `showBoard()`: prints the grid with 3x3 separators
- `run()`: main menu -> new game / load / instructions / exit
- `playGameLoop()`: reads commands, calls `GameController`
- Declares a win once `GameController::isSolved()` is true

```
int v = board.at(row,col)
        .getValue();
cout << (v == 0 ? '.' : ('0'+v))
      << " ";

if (col != 0 && col % 3 == 0)
    cout << "| ";
```

TEST CASE

INPUT

Command: "move 0 0 5"
Command: "undo"
Command: "solve"

EXPECTED OUTPUT

Board updates, cell (0,0) = 5
Last move reverted, board redraws
Board fully filled, win message shown

You Built a Complete OOP + DSA Project

- 8 classes, each with a single responsibility
- A backtracking algorithm you implemented twice (solve + generate)
- Two real data structures in action: stacks (undo/redo) and sets (candidates)
- Every method traced back to a requirement in the SRS

Next: play a full game end-to-end, then consider a GUI extension.

